

settings (Bhardwaj & Poria, 2023). A promising direction could be to take advantage of collections of chats such as WildChat (Zhao et al., 2024) for ChatGPT and manually curate real-world instances of falsely refused benign prompts or users seeking harmful information.

The strong focus on English examples poses another significant limitation for estimating real-world robustness, as users may communicate in any language. However, multi-lingual evaluation is logistically difficult—especially in academic contexts. Firstly, constructing high-quality datasets requires authors to speak the languages of the benchmark. Secondly, judge models are mostly trained and evaluated on English data, making automated evaluation more difficult. Many scenarios also rely on manual evaluations (e.g., monitoring outputs during development or verifying decisions by automated judges), which become impractical as additional languages are involved. Lastly, not all models are trained on the same set of languages, further complicating fair comparisons. We are not sure how to best address these problems; however, it would certainly be valuable to conduct transfer experiments to evaluate how well results generalize to languages that were not part of the model’s safety training.

Ill-defined requirements lead to datasets that measure alignment with inconsistent targets. It is important to remember that universally valid definitions of harmfulness do not exist, as ethical guidelines and societal norms vary across communities. Therefore, rather than attempting to make models “harmless”, the core *technical* goal of alignment research can only be making models adhere to the goals and values the model developers aim to instill, which is orthogonal to the specification of the goals and values themselves. In contrast, red-teaming and jailbreaking efforts deliberately attempt to push models to deviate from their intended behavior. In the context of robustness research, we argue that the community should focus on developing datasets that objectively assess how difficult it is to make models deviate from their intended behavior. Since it is infeasible to collect specific datasets according to each model developer’s guidelines, we think jailbreaking datasets should focus on only including content considered harmful by most, if not all, model developers. This ensures fair comparisons between models by different developers without penalizing model behavior that a developer opts not to restrict. While this may seem somewhat trivial, even widely used datasets like AdvBench (Zou et al., 2023b), which explicitly aim for this goal, include ambiguous content—such as drug-related questions that are only illegal in some jurisdictions and may not be considered harmful by all developers. These inconsistencies introduce noise and bias into evaluations, undermining our ability to fairly compare alignment strategies used by different model providers. Recent efforts to rigorously aggregate common guidelines from multiple developers before collecting prompts, such as in StrongRE-

JECT (Souly et al., 2024), represent a sensible direction and should be adopted by new datasets in the future as well.

3. Algorithms

Key Issues with Current Algorithms

Inconsistent implementation details, varying attack budgets, rigid-optimization objectives, and overlooked safety-overrefusal trade-off.

→ Unfair, noisy & biased comparisons.

Implementation details matter. Comparing algorithms and models across publications is complicated by implementation details that are often not clearly communicated, but introduce small errors, which can compound into significant discrepancies. Key issues include quantization, initialization, inconsistent use of chat templates and system messages, handling of whitespace tokens, as well as insufficient filtering of special tokens. Such discrepancies can affect attack success rates and were also observed in LLM capability evaluations (Biderman et al., 2024). These problems are especially relevant for white box attacks that rely on explicitly forcing a specific target token sequence, and are particularly difficult to catch, as finding them typically requires tedious manual verification for each model and implementation without raising obvious errors.

Models strongly expect whitespace characters and control tokens to appear at certain positions, such as before or after control tokens. If the target tokens are misaligned, the optimizer has to spend additional “effort” to move probability mass away from white space tokens, which is significantly more difficult in comparison to standard text tokens. Figure 2 empirically demonstrates this effect, showing that incorrectly accounting for whitespace tokens can reduce attack success rate by 28%. Appendix A.2 contains a more detailed discussion of this experiment. These issues can dramatically affect the success of an attack, and detecting them can be very tedious, as many whitespace characters are removed during detokenization and are thus invisible in the generated text. **We therefore urge developers to manually check the alignment of model generations with the target sequence in token-space to ensure correct alignment.**

The community widely agrees that token-level discrete optimization algorithms should exclude special tokens (e.g., instruction, tool use, role tokens) from the optimization. We thus echo the sentiment from (Thompson & Sklar, 2024b) and **strongly recommend that discrete attack algorithms track all state as a string** a user could pass directly to the model in a chat setting. While this may incur additional implementation overhead, it ensures that the discovered attacks are usable in practice. We find that even widely used implementations in nanoGCG (Zou et al., 2023b) and HarmBench (Mazeika et al., 2024) often fail to correctly filter out special tokens and other impossible attacks.

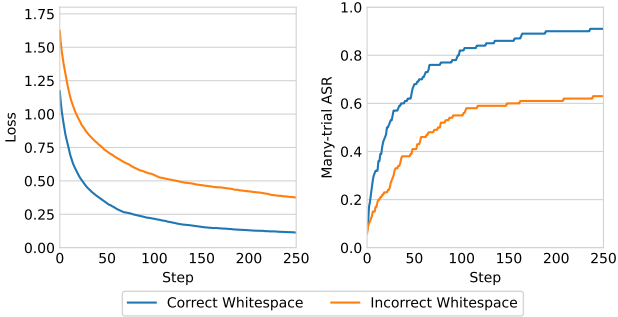


Figure 2: Impact of the SentencePiece white space token on GCG performance against Llama-2-7B-chat-hf. We judge the generated prompt at each step and report “many-trial” or “cumulative” ASR.

Chat Template	Filter	Dtype	ASR
Meta	Strict	BF16	0.77
HuggingFace	Strict	BF16	0.85
Meta	nanoGCG	BF16	0.78
Meta	Allow Non-ASCII	BF16	0.73
Meta	Strict	Int8	0.71
No Sys Msg	Strict	BF16	0.47

Table 1: Many-trial ASR of GCG against Llama-3.1-8B-Instruct with minor implementation changes. Even among reasonable configurations with system prompt, ASR varies significantly between 71% and 85%. Additional details can be found in Appendix A.3.

Table 1 illustrates these challenges. Given their widespread use and relatively well-supported software, we select GCG as the attack and Llama-3.1-8B-Instruct as the victim model, though other setups may exhibit even greater discrepancies. We compare several versions of the chat template, system prompts, quantization levels, and token filtering approaches. All reported setups are genuine best-effort implementations a researcher may attempt. Using the Meta-recommended [chat template](#) with a corrected strict token filtering procedure *lowers ASR by 8%* compared to the default template in the HuggingFace Transformers library with default token filtering from the official GCG implementation, while an int8 quantized model has *14% lower ASR*.

Efforts like (Mazeika et al., 2024; Chao et al., 2024) aim to tackle some of these issues by standardizing attack implementations and other aspects of the pipeline. However, in practice, many new algorithms are still built in bespoke repositories (Andriushchenko et al., 2024; Sadasivan et al., 2024), without taking advantage of these toolboxes. In our experience, the main reasons researchers opt for this are a lack of flexibility, reliability issues, and overly complex frameworks that attempt to abstract across models, but end up obfuscating underlying issues, leading to a steep learning curve and making them frustrating to work with. Furthermore, long-term maintenance is often not guaranteed, reducing the incentive for authors to implement their

method in a particular framework. Solving these problems is challenging, as the development and continued maintenance of such frameworks are not sufficiently recognized by the academic community to justify the sustained effort.

Attacks are not compared at consistent budgets. Many algorithms have hyperparameters that trade-off compute, perplexity, or the number of model queries for higher ASR. However, most papers report performance for a specific “operating point”—typically one that maximizes ASR for the available resources—without adequately illustrating the compute-ASR trade-off for their method and baselines. This narrow focus on final ASR leads to misleading comparisons. For example, GCG is usually run with default hyperparameters, which use a large number of steps, making it appear inefficient and compute-intensive (Chao et al., 2023; Sadasivan et al., 2024). However, as seen in Figure 4 and Appendix A.3, GCG can reach significant ASRs and a better compute-ASR trade-off with far fewer steps.

Thus, to ensure fair evaluations, attacks should be compared under controlled conditions by fixing either the attack budget or the success rate. Results where neither is matched often do not allow for meaningful comparisons unless one method Pareto-dominates others in all domains.

Some recent works take steps toward better standardization by reporting the mean number of queries per success (Chao et al., 2023), iso-GPU-time results (Sadasivan et al., 2024), wall-time on consistent hardware (Geisler et al., 2024), monetary cost estimates for commercial models (Hughes et al., 2024), or FLOP-budgets to quantify compute use (Boreiko et al., 2024). We think these are all valuable ideas, and **reporting and controlling for attack budget should become standard in the field**.

Optimization objectives are biased, unnatural, and poor proxies for attack success. Most competitive attacks (AmpleGCG, AutoDAN, BEAST, GCG, PGD, PAIR, ...) rely on predefined target string sequences provided by the jailbreak datasets. This objective is suboptimal as it is not model-agnostic. Nearly all targets follow a rigid “Sure, here...” template. When a model’s “natural” affirmative responses follow a different structure, the rigid targets become harder to optimize against and thus appear more robust than they actually are (Zhu et al., 2024) (c.f. Appendix A.4). A similar issue can occur when defenses are trained with knowledge of the optimization targets the attacks will use. This data leakage can lead to methods that defend remarkably well when attacks try to optimize for the same targets used during training; however, when attacks choose a different objective, they can quickly become more brittle. For instance, R2D2 (Mazeika et al., 2024) was adversarially trained against GCG, and its safety generalizes relatively well to other approaches that aim to optimize the datasets’ target sequence (e.g., AutoDAN). However, R2D2 remains

susceptible to PAIR (Chao et al., 2023), which employs LLM-driven optimization of the prompt rather than heavily relying on the explicit target sequence.

Furthermore, successfully manipulating a model into outputting the full target sequence does not guarantee a harmful response as models can “recover” and continue with a safe response. Instead, the effectiveness of the generated attack depends on the implicit bias of the optimization algorithm. While some methods attempt to diversify the targets (Jia et al., 2024; Zhu et al., 2024; Thompson & Sklar, 2024b) to speed up the optimization process, we think the field could benefit from considering a broader perspective. In other domains, such as computer vision, adversarial attacks were able to make progress by constructing consistent loss functions, where lower loss is guaranteed to correspond to higher success rates (Tramer et al., 2020).

Although it may not be feasible to design fully consistent loss functions for generative models, we think **the community should explore ways to more directly optimize for successful attacks**, rather than overfocusing on optimizing for specific token sequences that may introduce bias and fail to provide a consistent surrogate signal for ASR.

Defenses do not consistently report the safety-overrefusal trade-off. Designing safe LLMs inherently requires balancing competing objectives: maximizing robustness to adversarial inputs while minimizing overrefusal of benign prompts. This trade-off is well documented (Thompson & Sklar, 2024a; An et al., 2024), yet, with few exceptions (Yu et al., 2024; Dubey et al., 2024; Guan et al., 2024), existing defenses rarely account for it comprehensively. Many defenses only include results for capability benchmarks but do not use overrefusal datasets such as the aforementioned XSTest (Röttger et al., 2023) or the CoCoNot contrast set (Brahman et al., 2024). We argue that **safety and overrefusal should always be evaluated in tandem**, with methodologies explicitly acknowledging this trade-off using standard benchmarks.

4. Evaluation

The last stage of the LLM safety evaluation pipeline typically consists of sampling a model generation and judging whether the model has been broken or not.

Key Issues with Current Evaluation

Limited to greedy generation, a fragmented ecosystem, and judges with model- & attack-specific biases.

→ Overoptimistic results, misleading & biased evaluations for untested attacks & defenses.

Greedy generation is unrealistic and ignores the distributional nature of LLM outputs. Greedy decoding, where

the model always selects the highest-probability token at each step, is currently the dominant setup for sampling from victim models in the context of LLM robustness. While convenient and deterministic, this strategy does not match practical chatbot settings where model responses are sampled using a variety of methods (Vijayakumar et al., 2016; Holtzman et al., 2019; Su et al., 2022) to avoid repetitive answers and produce natural output.

In fact, even with a fixed prompt, frontier models can often be broken by simply sampling a large number of generations at temperature 1 (Hughes et al., 2024)².

From a statistical standpoint, evaluating models at temperatures that differ from deployment settings is undesirable, as it can make noise mitigation techniques like resampling harder or even impossible in the case of greedy decoding (Miller, 2024). Additionally, Scholten et al. (2024) show that this setup fails to accurately characterize real-world model behavior, leading to over-optimistic bounds on safety and alignment: models that appear robust in greedy generation may still produce harmful content a significant fraction of the time when deployed with more realistic sampling strategies. Similar limitations also apply from the attacker’s perspective: an attack prompt effective for greedy generation may not reliably generalize to real-world settings with more representative sampling methods (Hughes et al., 2024). To better assess the “robustness” of individual attack prompts and to provide better model safety guarantees, **we think a more realistic, distributional approach which samples multiple generations under realistic conditions and judges them independently may offer new insights.**

Single-trial and many-trial jailbreaks lack clear differentiation. There is a disconnect between methods that

perform multiple queries and evaluate multiple model responses (“many-trial”) until a jailbreak is found or the search budget is exhausted (Hughes et al., 2024; Liao & Sun, 2024; Chao et al., 2023; Andriushchenko et al., 2024), and those which optimize a prompt first and then test only the optimized prompt using greedy generation (“single-trial”) (Zou et al., 2023b; Geisler et al., 2024; Sadasivan et al., 2024; Liu et al., 2023a). Comparing these attack types is complicated as their practicality and efficiency depend on the model defenses employed and the cost model of the attacker. For example, cheaply generating many prompt candidates with low individual success probability may be efficient from a time or dollar cost standpoint but would require many model rollouts and could be more easily filtered by providers due to the suspicious query pattern of sampling a large amount of very similar prompts. In contrast, methods that expend more compute to come up with fewer (or even a single) prompts with high individual success likelihood may be

²Hughes et al. (2024) primarily focus on augmented versions of harmful prompts, but an ablation using a fixed prompt shows that simply sampling more generations consistently increases ASR.